

模块 6：搜索策略 — 从贪心到 MCTS

模块概述

核心问题：当前编程 Agent 的搜索策略处于什么水平？未来在哪里？

这是本课程的理论核心模块。我们将从一个根本性的观察出发：当前几乎所有主流编程 Agent（Cursor、Copilot、Devin、OpenHands）都在使用最朴素的贪心搜索。它们一步步前进，从不回头，遇到死路就靠 LLM 的“直觉”硬闯。

这像什么？这像一个围棋选手，每步只看当前局面选“最好的一手”，从不进行任何前瞻性思考。

我们知道这样的棋手会被 AlphaGo 碾压。那么——编程 Agent 的 AlphaGo 时刻何时到来？

本模块将：1. 绘制搜索策略的完整进化光谱 2. 分析贪心策略为何在工程实践中占据主导地位 3. 深入解剖 AlphaGo 的 MCTS 架构，并映射到代码场景 4. 揭示代码空间搜索的特殊困难 5. 追踪 LATS、SWE-Search、CodeTree 等前沿进展 6. 推演“编程 Agent 的 AlphaGo 时刻”需要哪些条件成熟

前置知识：树搜索基础（BFS/DFS）、强化学习基本概念、Module 3-5 中对 ReAct/Reflexion/ToT 的了解。

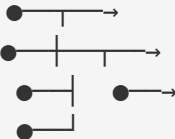
6.1 搜索策略进化光谱

四级搜索策略

编程 Agent 的搜索策略可以按照“探索能力”由低到高排列成一个光谱：

搜索策略进化光谱

Level 0		Level 1		Level 2		Level 3
贪心搜索	→	线性回溯	→	树搜索	→	蒙特卡洛树搜索
Greedy		Linear		Tree Search		MCTS
(ReAct)		Backtrack		(ToT)		(AlphaGo-style)
		(Reflexion)				

			
只前进 不回头	前进+后退 单条路径	分支探索	统计引导搜索
探索能力: ★☆☆☆☆	★★☆☆☆	★★★☆☆	★★★★★
计算成本: \$	\$\$	\$\$\$	\$\$\$\$
延迟: 秒级	分钟级	分钟~小时	小时级
代表性工作: ReAct (Yao 2023)	Reflexion (Shinn 2023)	Tree of Thoughts (Yao 2023)	LATS (Zhou 2023) SWE-Search (Antonis 2024)
当前主流程 Agent 所在位置: ▲ 这里 (Level 0)			

各级详解

Level 0: 贪心搜索 (Greedy Search)

机制：Agent 在每一步选择当前看起来最优的 action，执行后不回溯。

代表：ReAct (Yao et al., 2023)。几乎所有商业编程 Agent 的核心 loop：

```
Observe → Think → Act → Observe → Think → Act → ... → Done/Fail
```

特征： - 单条执行路径，无分支 - 依赖 LLM 的“一次性判断”质量 - 遇到错误时，在同一条路径上尝试修复（不是真正的回溯） - 计算成本最低：每步 1 次 LLM 调用

编程 Agent 中的表现：

```
# 伪代码：贪心编程 Agent
def greedy_agent(task):
    state = initial_state(task)
    for step in range(max_steps):
        action = llm.generate_next_action(state) # 贪心选择
        state = execute(action, state)
        if is_solved(state):
            return state
    return FAILED
```

局限：一旦在早期做出错误决策（如选错了要修改的文件），后续所有努力都可能浪费。

Level 1：线性回溯（Linear Backtracking）

机制：Agent 执行一条路径后，如果失败则进行反思（reflection），然后用反思结果指导重试。

代表：Reflexion（Shinn et al., 2023）。

```
Attempt 1: Act → Act → Act → Fail
                                     ↓ (reflect)
Attempt 2: Act → Act → Act → Fail
                                     ↓ (reflect, with memory of attempt 1-2)
Attempt 3: Act → Act → Act → Success!
```

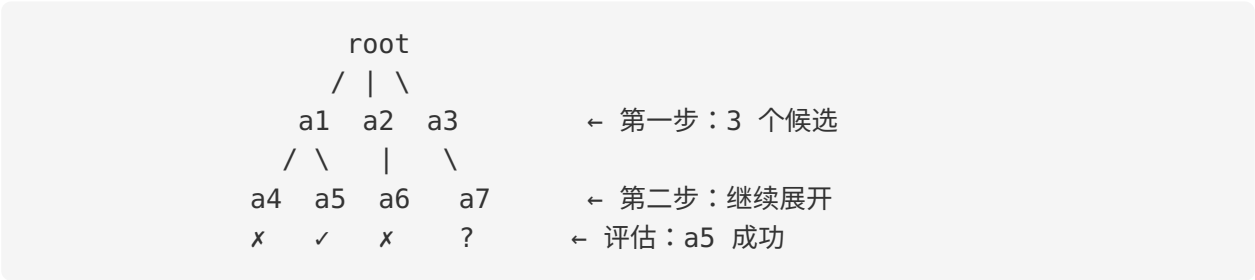
特征：- 本质上是“串行重试 + 经验积累” - 每次重试利用之前失败的经验 - 仍然是线性的——不会同时探索多条路径 - 计算成本： $O(n)$ 次完整尝试，其中 n 是重试次数

编程 Agent 中的表现：当 Cursor/Copilot 生成的代码编译失败时，它们会看到错误信息并“重试”。这本质上就是 Level 1，只不过反思的质量参差不齐。

Level 2：树搜索（Tree Search）

机制：在每个决策点生成多个候选 action，形成搜索树，通过某种策略（BFS/DFS/beam search）在树上搜索。

代表：Tree of Thoughts（Yao et al., 2023）。



特征：- 显式维护多条探索路径 - 需要评估函数（evaluator）来判断哪些分支更有前途 - 计算成本陡增：分支因子 b ，深度 d ，最坏情况 $O(b^d)$ - 需要状态管理：如何“回退”到之前的代码状态

Level 3：蒙特卡洛树搜索（MCTS）

机制：结合树搜索和随机模拟。用统计方法（UCB）平衡探索与利用，通过大量 simulation 估算每个分支的价值。

代表：LATS（Zhou et al., 2023）、SWE-Search（Antonis et al., 2024）。

特征： - 不需要穷举搜索树——用采样代替 - UCB 公式自动平衡“挖掘已知好路径”和“探索未知路径” - 需要快速的 simulation/evaluation 函数 - 计算成本高但可控： $O(n)$ 次 simulation， n 由预算决定

关键区别：Level 2 像“看得到所有路的迷宫”，Level 3 像“在黑暗迷宫中用统计方法找路”——后者更适合超大搜索空间。

为什么这个光谱重要？

维度	Level 0	Level 1	Level 2	Level 3
SWE-bench Lite 典型表现	~30-45%	~40-55%	~50-60%	~55-65%+
每任务 Token 消耗	10K-50K	50K-200K	100K-500K	500K-2M+
延迟	30s-3min	3-15min	10-60min	1-6h
工程复杂度	低	中	高	极高
当前商业可行性	□ 主流	□ 部分采用	△ 实验性	□ 研究阶段

洞察：搜索策略的每一级提升都带来性能收益，但也带来指数级的成本增长。这个 trade-off 是理解当前编程 Agent 格局的关键。

6.2 当前主流为什么是贪心

既然更高级的搜索策略明显更强，为什么工业界几乎清一色选择 Level 0？这不是因为工程师不懂树搜索——而是因为存在一系列强大的现实约束。

6.2.1 Token 经济学

这是最直接的约束。以 GPT-4o 级别模型为例：

单次贪心尝试： ~30K tokens ≈ \$0.15

Level 1 (3 次重试)： ~100K tokens ≈ \$0.50

Level 2 (分支因子 3, 深度 4): ~500K tokens ≈ \$2.50
Level 3 (500 次 simulation): ~2M tokens ≈ \$10.00

每天处理 1000 个任务的成本:
Level 0: \$150/天
Level 1: \$500/天
Level 2: \$2,500/天
Level 3: \$10,000/天

对于一个拥有百万用户的产品（如 Cursor），从 Level 0 升级到 Level 3 意味着**基础设施成本增加近 100 倍**。在模型推理成本持续但缓慢下降的今天，这是一道简单但残酷的算术题。

6.2.2 延迟约束

用户体验对延迟极为敏感：

用户心理预期：

< 10s	"即时响应"	→ 用户满意度高
10-60s	"可接受等待"	→ 用户愿意等
1-5min	"后台处理"	→ 需要明确进度反馈
5-30min	"异步任务"	→ 用户切换到其他工作
> 30min	"批处理"	→ 用户质疑是否值得等

各策略典型延迟：

Level 0 (Greedy): 30s - 3min ← 在"可接受"范围内
Level 1 (Retry): 3 - 15min ← 勉强接受
Level 2 (Tree): 10 - 60min ← 需要异步化
Level 3 (MCTS): 1 - 6h ← 必须批处理

关键洞察：编程 Agent 的主流使用场景是“交互式辅助”（interactive copilot），而不是“离线批处理”。在交互场景下，延迟的优先级可能高于准确率——一个 3 秒给出 70% 正确答案的 Agent 往往比一个 30 分钟给出 90% 正确答案的 Agent 更受欢迎。

6.2.3 状态管理复杂度

这是一个被低估但极为重要的工程障碍。

在围棋中，状态是一个 19×19 的棋盘，复制和回退成本几乎为零。但在代码中：

```
代码状态 = {  
    文件系统快照,           # 可能涉及数百个文件
```

```
    运行中的进程状态,      # 开发服务器、数据库
    环境变量,              # 配置信息
    安装的依赖版本,        # node_modules, venv
    外部服务状态,          # API 调用的副作用
    Git 历史,              # 已提交的变更
}
```

要进行树搜索，你需要能够：1. **快照 (Snapshot)**：保存当前完整状态 2. **分支 (Branch)**：创建独立的探索路径 3. **回退 (Rollback)**：恢复到之前的状态

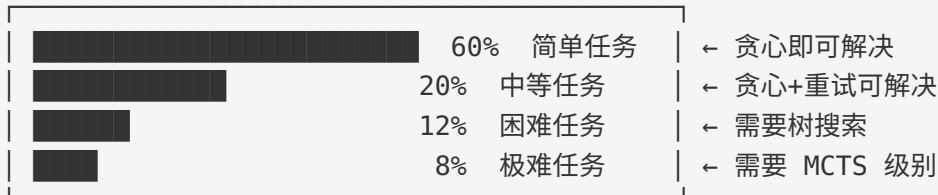
对于纯文件修改，Git 的 stash/branch 机制可以近似解决。但如果 Agent 的操作涉及：- 运行了一个修改数据库的迁移脚本 - 安装或卸载了依赖包 - 调用了外部 API（发送了邮件、创建了资源）

这些操作的“撤销”成本极高，有些甚至不可逆。这使得树搜索在实际编程场景中远比在围棋中复杂。

6.2.4 “够用就好”原则

贪心策略虽然简单，但在很多场景下确实够用：

任务难度分布（估计）：



贪心策略的“性价比”：

- 解决了 ~80% 的实际编程任务
- 成本是 MCTS 的 1/50
- 延迟是 MCTS 的 1/100
- 工程复杂度是 MCTS 的 1/10

对于商业产品，这个 trade-off 非常清晰：用 Level 0 覆盖 80% 的任务，剩下 20% 交给人类处理，远比投资 Level 3 去覆盖额外 15% 更经济。

6.2.5 小结：贪心的统治地位

贪心策略的统治不是因为它最优，而是因为它在**当前约束条件下**最实用：

约束	影响
Token 成本	限制了每次交互的计算预算
延迟要求	排除了需要大量搜索的策略
状态管理	增加了分支/回退的工程成本
80/20 法则	降低了升级搜索策略的边际收益

但这些约束正在松动。Token 成本每年下降 5-10 倍，异步 Agent（如 Devin）正在改变延迟预期，容器化技术在简化状态管理。这就是为什么我们需要理解更高级的搜索策略——它们可能比我们预期的更快到来。

6.3 AlphaGo 深入解剖

要理解编程 Agent 搜索的未来，我们必须回到 2016 年那个改变 AI 历史的系统：AlphaGo。它的核心不是某个单一技术，而是一个精巧的搜索系统。

6.3.1 MCTS 四步循环

MCTS 是一个迭代算法，每次迭代包含四个步骤：

MCTS 一次迭代

Step 1: Selection (选择)
从根节点沿着"最有前途"的路径向下选择

[R]
/ | \
[A] [B] [C] ← UCB 选 B
|
[D] ← UCB 选 D
↓
到达叶节点

Step 2: Expansion (扩展)
在叶节点生成新的子节点

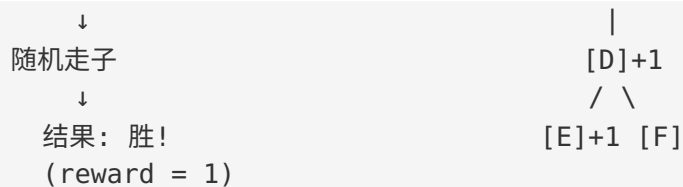
[R]
/ | \
[A] [B] [C]
|
[D]
/ \
[E] [F] ← 新节点

Step 3: Simulation (模拟)
从新节点进行快速模拟直到终局

[E]
↓
随机走子

Step 4: Backpropagation (反向传播)
将模拟结果沿路径反向更新所有祖先节点

[R] +1
/ | \
[A] [B]+1 [C]



UCB (Upper Confidence Bound) 公式

Selection 步骤的核心是 UCB 公式，它平衡 exploitation 与 exploration:

$$\text{UCB}(\text{node}) = \underbrace{Q(\text{node})/N(\text{node})}_{\text{exploitation 项}} + c * \underbrace{\sqrt{\ln(N(\text{parent})) / N(\text{node})}}_{\text{exploration 项}}$$

exploitation 项
(平均奖励：
越高越好)

exploration 项
(访问越少，
探索价值越大)

其中：

$Q(\text{node})$ = 该节点累积奖励

$N(\text{node})$ = 该节点被访问次数

$N(\text{parent})$ = 父节点被访问次数

c = 探索常数 (通常 $\sqrt{2} \approx 1.414$)

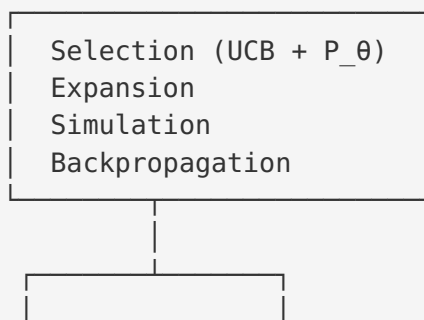
直觉：UCB 就像一个投资策略——既投资已知高回报的项目 (exploitation)，也投资一些尝试不多但可能有惊喜的项目 (exploration)。随着采样增加，统计估计越来越准，搜索自然收敛到最优路径。

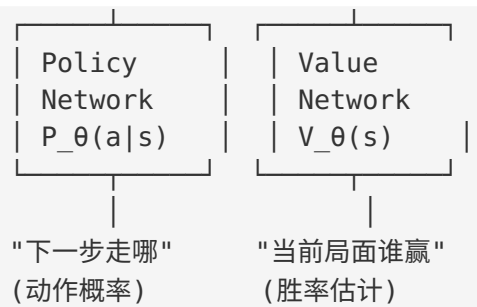
6.3.2 AlphaGo 的架构

AlphaGo 不是纯 MCTS，而是 MCTS + 深度神经网络的组合：

AlphaGo 架构

MCTS





三个核心组件的协作：

1. Policy Network: 给 MCTS 的 Selection 提供先验概率
→ 引导搜索优先探索"看起来合理"的走法
2. Value Network: 替代 Simulation 中的随机走子
→ 直接估计局面胜率，不需要走到终局
3. MCTS: 整合两个网络的输出，通过统计搜索找到最优走法

关键创新： - **Policy Network 替代均匀随机：**传统 MCTS 的 Selection 对所有子节点一视同仁，AlphaGo 用 policy network 给出先验概率，大幅减少需要探索的分支 - **Value Network 替代 rollout：**传统 MCTS 需要随机模拟到终局，AlphaGo 用 value network 直接评估中间局面，大幅减少模拟深度 - **自我对弈 (Self-Play) 训练：**policy 和 value network 通过自我对弈不断提升，形成正向循环

6.3.3 映射到代码场景

现在，让我们将 AlphaGo 的每个组件映射到编程 Agent 的代码搜索场景：

AlphaGo ↔ 编程 Agent 映射

AlphaGo 组件	编程 Agent 对应物
棋盘状态 (19×19)	→ 代码仓库当前状态 (文件系统 + Git 历史)
合法走法 (~250)	→ 可能的代码修改 (理论上无限，实际可生成 3-10 个候选)
Policy Network	→ LLM 生成候选修改 (类似 $P(\text{action} \text{state})$)
Value Network	→ 代码质量评估器 (测试通过率 + lint + LLM 打分)
Simulation/Rollout	→ 快速评估: 运行测试子集 / 静态分析 / LLM 快速判断
终局胜负判定	→ 所有测试通过 + 代码审查通过

Selection 在代码中的实现

```
# 代码场景中的 UCB Selection
def select_next_edit(node, c=1.414):
    """选择下一步要探索的代码修改"""
    best_score = -inf
    best_child = None
    for child in node.children:
        if child.visits == 0:
            return child # 未访问过的节点优先
        exploit = child.total_reward / child.visits
        explore = c * sqrt(log(node.visits) / child.visits)
        # LLM 先验加成 (类似 AlphaGo 的 policy network)
        prior = child.llm_prior_probability
        score = exploit + explore + prior
        if score > best_score:
            best_score = score
            best_child = child
    return best_child
```

Expansion 在代码中的实现

```
# 代码场景中的 Expansion
def expand_node(node, llm, num_candidates=3):
    """在当前代码状态上生成候选修改"""
    state = node.code_state
    prompt = f"""
    当前代码状态: {state.description}
    任务目标: {state.task}
    已尝试过的方法: {state.tried_approaches}

    请生成 {num_candidates} 个不同的修改方案:
    """
    candidates = llm.generate(prompt, n=num_candidates)
    for i, candidate in enumerate(candidates):
        child = TreeNode(
            code_state=apply_edit(state, candidate),
            edit=candidate,
            llm_prior_probability=candidate.confidence,
            parent=node
        )
        node.children.append(child)
```

Simulation 在代码中的实现

```
# 代码场景中的 Simulation
def simulate(node):
    """快速评估一个代码状态的质量"""
    state = node.code_state
    scores = []

    # 信号 1: 语法/编译检查 (最快, ~1s)
    compile_ok = run_syntax_check(state)
    scores.append(1.0 if compile_ok else 0.0)
    if not compile_ok:
        return 0.0 # 编译都不过, 直接返回 0

    # 信号 2: 快速测试子集 (~5-10s)
    test_result = run_fast_tests(state, subset="relevant")
    scores.append(test_result.pass_rate)

    # 信号 3: 静态分析 (~2s)
    lint_score = run_linter(state)
    scores.append(lint_score)

    # 信号 4: LLM 快速评估 (~3s)
    llm_score = llm_quick_eval(state, task=state.task)
    scores.append(llm_score)

    return weighted_average(scores, weights=[0.2, 0.4, 0.1, 0.3])
```

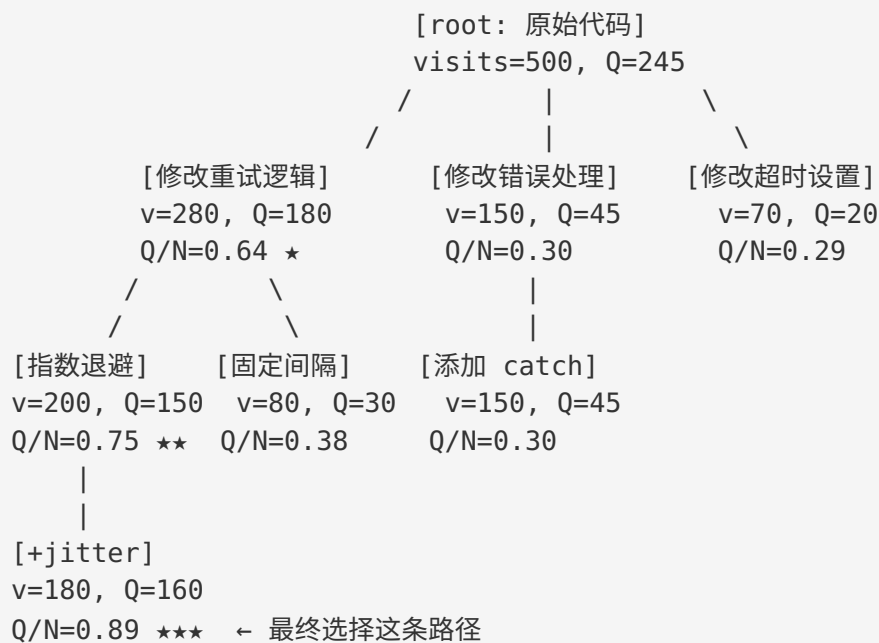
Backpropagation 在代码中的实现

```
# 代码场景中的 Backpropagation
def backpropagate(node, reward):
    """将评估结果沿路径向上传播"""
    current = node
    while current is not None:
        current.visits += 1
        current.total_reward += reward
        current = current.parent
    # 更新后, 该路径上所有节点的 Q/N 值都被更新
    # 下次 Selection 时会自动偏向更高价值的分支
```

6.3.4 一个完整的代码 MCTS 示例

任务: 修复一个 HTTP 请求重试逻辑的 bug

MCTS 搜索树 (500 次迭代后):



对比贪心策略：贪心 Agent 可能一上来就选了“修改错误处理”（因为错误信息最明显），浪费大量 token 后才发现根因在重试逻辑。MCTS 通过统计采样，在 500 次迭代中自动发现“指数退避 + jitter”是最优解。

6.4 代码空间搜索的特殊困难

理论上，将 MCTS 应用到代码生成应该很直接。但实际上，代码空间相比围棋有一系列根本性的困难，使得直接迁移 AlphaGo 的方法面临巨大挑战。

6.4.1 无界状态空间

围棋：

- 棋盘大小： $19 \times 19 = 361$ 个位置
- 状态数上界： $3^{361} \approx 10^{172}$ (每个位置：黑/白/空)
- 虽然巨大，但有明确的上界

代码：

- 文件数量：无上界
- 每个文件长度：无上界

- 可能的编辑：在任意位置插入/删除/修改任意字符
- 状态空间：本质上是无限的

类比：围棋是在一个固定大小的房间里搜索，
代码是在一个不断扩张的宇宙中搜索。

实际影响：这意味着我们无法像围棋那样对状态空间进行完整的表征（representation）。编程 Agent 的 MCTS 必须在一个**开放的、连续增长**的状态空间中操作。

6.4.2 极高的分支因子

围棋的分支因子：

- 平均 ~250 个合法走法（19×19 空位）
- AlphaGo 的 Policy Network 将有效分支因子降至 ~10-20

代码的分支因子：

- 在一个 1000 行的文件中：
 - 可以修改任意一行（1000 个位置）
 - 每个位置可以进行无数种修改
 - 可以添加新行（1001 个插入点）
 - 可以删除任意行组合（ 2^{1000} 种）
- 理论分支因子：实际上是无限的

LLM 作为 Policy Network 的降维效果：

- LLM 通常生成 3-10 个“合理”的候选修改
- 有效分支因子：~5-10
- 但：LLM 的候选可能遗漏正确答案！（围棋的 Policy Network 不会）

核心问题：在围棋中，policy network 只需要在 ~250 个合法走法中排序。在代码中，LLM 需要从**无限可能中生成**合理的候选。这是“排序问题”和“生成问题”的本质区别——后者难得多。

6.4.3 模糊的评估函数

这可能是最根本的困难。AlphaGo 的成功高度依赖于准确的 value network——给定一个局面，它能以很高的精度估计胜率。

围棋的评估：

终局判定：100% 准确（数子即可）	
Value Network：~80% 准确	
结合 MCTS：>99% 准确	

Ground truth 完全可获得

代码的评估：

测试通过率：取决于测试覆盖率

- 好的测试套件：~70% 可靠
- 差的测试套件：~30% 可靠
- 没有测试：0%

Lint/静态分析：~20% 可靠
(只能检查表面问题)

LLM-as-Judge：~60% 可靠
(可能与生成 LLM 犯同样的错)

Ground truth 通常不可获得
(什么是"正确"的代码?)

Rice's Theorem 的阴影：从理论计算机科学的角度，判断“一段代码是否满足某个非平凡规约”是不可判定的（Rice's Theorem）。这意味着**不存在完美的代码评估函数**——这是一个根本性的理论限制，围棋不存在这个问题。

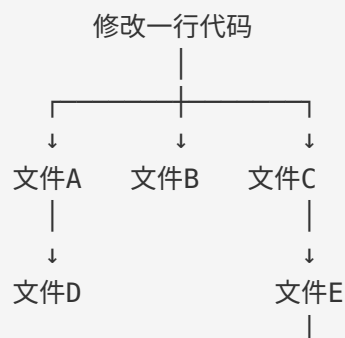
6.4.4 非局部效应

围棋：

- 落一子主要影响周围几步范围
- 偶有全局性的"打劫"/"大龙"
- 但大部分局部变化不会改变远处的判断

代码：

- 修改一个函数签名 → 所有调用者必须更新
- 修改一个类型定义 → 整个类型系统级联变化
- 修改一个配置值 → 运行时行为全面改变
- 修改一个依赖版本 → 不可预测的兼容性问题



↓
文件F ← 6 层传播后出现 bug

这意味着 MCTS 中的 Simulation 步骤要准确评估一个代码修改，可能需要考虑整个代码库的状态——这远比评估一个围棋局面昂贵。

6.4.5 不可逆性

围棋：

- 理论上可以"悔棋"（在搜索中）
- 棋盘状态的保存和恢复成本为 $O(1)$
- 落子没有副作用

代码：

- 文件修改可以通过 Git 回退 ✓
- 但以下操作不可逆：
 - ✗ 已执行的数据库迁移
 - ✗ 已发送的 API 请求
 - ✗ 已安装/卸载的系统级包
 - ✗ 已杀死的进程
 - ✗ 已消耗的 API 配额

6.4.6 困难汇总

维度	围棋	代码	困难倍增
状态空间大小	$\sim 10^{172}$	∞	∞
分支因子	~ 250	∞ (LLM 降至 $\sim 5-10$)	$\sim 1x$ (但生成 vs 排序)
评估函数精度	$\sim 99\%$ (终局)	$\sim 50-70\%$ (测试+LLM)	$\sim 2x$
状态回退成本	$O(1)$	$O(1) \sim O(\infty)$	视操作而定
单步效应范围	局部	全局	$\sim 10x$
理论可判定性	可判定	不可判定 (Rice' s Theorem)	根本性

这些困难不是说 MCTS 在代码领域不可行——而是说它需要本质性的改造，不能简单套用围棋的框架。下一节我们将看到，研究者们正在如何应对这些挑战。

6.5 前沿进展

尽管存在诸多困难，学术界已经开始将树搜索和 MCTS 思想引入编程 Agent。以下是三个最具代表性的前沿工作。

6.5.1 LATS: Language Agent Tree Search

论文：Zhou et al., “Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models” (2023)

核心思想：将 LLM 同时用作 MCTS 的 policy network、value network 和 simulation engine。

LATS 架构

传统 MCTS:

Policy Net

Value Net

Simulation

→ 专用神经网络

→ 专用神经网络

→ 随机走子

LATS:

Policy Net

Value Net

Simulation

→ LLM (生成 action)

→ LLM (评估状态)

→ LLM (推理) + 环境反馈(测试等)

LATS 将 AlphaGo 需要的三个独立组件统一为一个 LLM + 环境反馈

算法流程：

LATS 一次迭代:

1. Selection: 用 UCB 选择最有前途的节点

2. Expansion: 用 LLM 生成 n 个候选 action

3. Evaluation: 对每个新节点:

a) 执行 action 获取环境反馈

b) 用 LLM 评估当前状态质量 (value)

c) 如果任务完成 → 返回成功

4. Simulation: 用 LLM 对有前途的节点进行 k 步前瞻

5. Backprop: 将 value 沿路径反向传播

6. Reflection: 失败的路径生成"反思"文本，作为后续搜索的上下文

关键创新： - **LLM 一体化：** 不需要训练单独的 policy/value 网络 - **自然语言反思：** 失败路径的经验以文本形式保留，类似 Reflexion - **环境反馈整合：** 测试结果、编译错误等直接作为评估信号

性能结果： - HumanEval（代码生成）：LATS 89.4% vs. ReAct 67.0% (↑ 22.4%) - WebShop（交互任务）：LATS 75.9% vs. ReAct 60.5% (↑ 15.4%) - 代价：token 消耗增加 ~10-50 倍

局限： - 计算成本很高，不适合实时交互 - LLM-as-value-network 的精度不如专用模型 - 在复杂代码任务上的验证还不充分

6.5.2 SWE-Search：面向软件工程的 MCTS

论文： Antonis et al., “SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement” (2024)

核心思想： 将 MCTS 专门适配到软件工程任务，解决代码空间搜索的特殊问题。

SWE-Search 架构

输入：SWE-bench 任务 (issue description + codebase)

搜索树的节点 = Agent 的"思考状态"

搜索树的边 = Agent 的一步操作 (搜索文件/阅读代码/编辑代码/运行测试)

[理解问题]

/ | \

[搜索文件A]

[搜索文件B]

[搜索文件C]

[阅读函数x]

[阅读函数y]

[编辑函数x]

[编辑函数y]

[运行测试]

[运行测试]

PASS

FAIL

[反思 + 修改]

[运行测试]

PASS

核心机制：

1. 每个节点包含 Agent 的完整"思考链"

2. LLM 作为 value function 评估每个状态

3. UCB 引导搜索方向

4. 失败路径的反思信息传递给相邻分支

关键设计决策：

- **粗粒度操作：**树的每一层不是单个代码字符的修改，而是一个完整的“Agent 操作”（搜索文件、编辑代码、运行测试）。这大幅降低了有效分支因子。
- **LLM 状态评估：**用 LLM 评估“当前探索状态距离解决问题还有多远”，作为 value function。
- **反思传播：**一个分支的失败经验会传递给同层其他分支，避免重复犯错。

SWE-bench 结果：

- SWE-bench Lite: SWE-Search 在 Moatless 框架上提升了 23% 的相对性能
- 对于“困难”问题的提升更为显著

与纯贪心的关键区别：

贪心 Agent 遇到困难问题：

Step 1: 搜索代码 → 找到文件 A → 编辑 → 测试失败
Step 2: 看到错误 → 继续修改文件 A → 测试仍然失败
Step 3: 继续修改... (陷入局部最优)
Step 4: 超时失败

SWE-Search 遇到同样的问题：

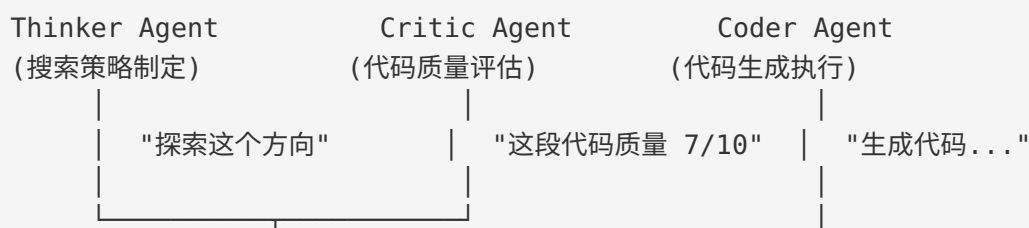
Branch 1: 搜索代码 → 找到文件 A → 编辑 → 测试失败 (value=0.3)
Branch 2: 搜索代码 → 找到文件 B → 编辑 → 测试部分通过 (value=0.6) ★
Branch 3: 搜索代码 → 找到文件 C → 编辑 → 编译失败 (value=0.1)
→ UCB 选择 Branch 2 继续深入探索
→ Branch 2.1: 进一步修改 → 所有测试通过! (value=1.0)

6.5.3 CodeTree：代码生成中的树搜索

论文：Li et al., “CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models” (2024)

核心思想：专门为代码生成任务设计的树搜索框架，引入“批评者 Agent”作为评估函数。

CodeTree 框架



搜索树构建

请求生成

决策流：

1. Thinker 分析任务，提出多个解题策略
2. Coder 根据策略生成候选代码
3. Critic 评估每个候选的质量
4. 基于评估结果，Thinker 决定：
 - 继续深入最佳分支？
 - 回溯尝试其他策略？
 - 停止搜索返回最佳结果？

创新点： - **多 Agent 协作搜索：** 将搜索分解为三个角色（策略、执行、评估），每个角色可以用不同的 LLM - **统一的代码搜索框架：** 将 BFS、DFS、Best-First Search 统一在一个框架下 - **与环境反馈的集成：** 将测试执行结果与 LLM 评估相结合

结果： - HumanEval: 95.1% (vs. 单次生成 ~85%) - MBPP: 82.7% (vs. 单次生成 ~75%) - CodeContests: 显著提升

6.5.4 前沿工作对比

维度	LATS	SWE-Search	CodeTree
发表时间	2023	2024	2024
搜索算法	MCTS	MCTS	BFS/DFS/Best-First
Policy Network	通用 LLM	通用 LLM	Thinker Agent
Value Network	LLM 评估	LLM 评估	Critic Agent
Simulation	LLM 推理	测试执行	测试 + LLM
目标场景	通用 Agent 任务	软件工程 (SWE-bench)	代码生成
状态管理	基本	中等 (Git-based)	基本
反思机制	□	□	□
多 Agent	□	□	□ (3 Agent)
Token 开销	~10-50x	~5-20x	~5-15x

维度	LATS	SWE-Search	CodeTree
主要创新	LLM 一体化 MCTS	工程化 MCTS	多 Agent 树搜索

6.5.5 什么在起作用，什么还不行

已经验证有效的： 1. **树搜索 > 贪心：** 在所有可比实验中，某种形式的树搜索都优于贪心策略 2. **环境反馈是关键：** 测试执行、编译检查等确定性反馈极大提升了搜索效率 3. **反思传播有用：** 一个分支的失败经验帮助其他分支避免重复错误 4. **LLM 是可用的 value function：** 虽不完美，但 LLM 评估提供了有意义的搜索引导

仍然面临的挑战： 1. **成本问题未解决：** 树搜索的 token 消耗仍然是贪心的 5-50 倍 2. **评估函数不够可靠：** LLM-as-judge 的精度远低于 AlphaGo 的 value network 3. **状态管理原始：** 当前系统主要依赖文本级别的状态管理，缺乏高效的代码快照/回退机制 4. **缺乏训练信号：** AlphaGo 通过自我对弈生成了数百万训练样本；代码领域缺乏类似的高效训练方法 5. **泛化性存疑：** 在 benchmark 上有效的搜索策略，是否能泛化到真实世界的软件工程任务？

6.6 “编程 Agent 的 AlphaGo 时刻”

2016 年 3 月，AlphaGo 击败李世石。那一刻标志着 AI 在一个被认为“需要人类直觉”的领域达到了超人水平。

编程领域的类似时刻何时到来？我们来分析需要成熟的条件。

6.6.1 条件一：更快、更便宜的推理



2025年: ~150 tokens/s
2026年: ~300+ tokens/s (推测)

分析: 如果推理成本以每年 3-5 倍的速度下降, 那么 2026-2027 年, 当前 MCTS 级别搜索的成本将降至今天贪心搜索的成本。这是最确定的趋势。

6.6.2 条件二: 更好的评估函数

评估函数是“编程 Agent 的 AlphaGo 时刻”最关键的瓶颈。需要多个方向共同推进:

评估函数改进路径

路径 1: 更好的测试生成

当前: 开发者写的测试 (覆盖率 ~30-60%)

未来: LLM 自动生成高覆盖率测试 → 更可靠的评估信号

路径 2: 形式化验证的普及

当前: 仅用于安全关键领域

未来: LLM 辅助生成形式化规约 → 数学级别的正确性保证

路径 3: 专用 Value Network

当前: 通用 LLM-as-Judge

未来: 在大量 (代码, 质量评分) 数据上训练的专用模型

类比: AlphaGo 的 value network 是专门训练的, 不是通用模型

路径 4: 多信号融合

当前: 单一信号 (测试 OR lint OR LLM)

未来: 融合测试 + 类型检查 + 性能 profile + 安全扫描 + LLM 评审

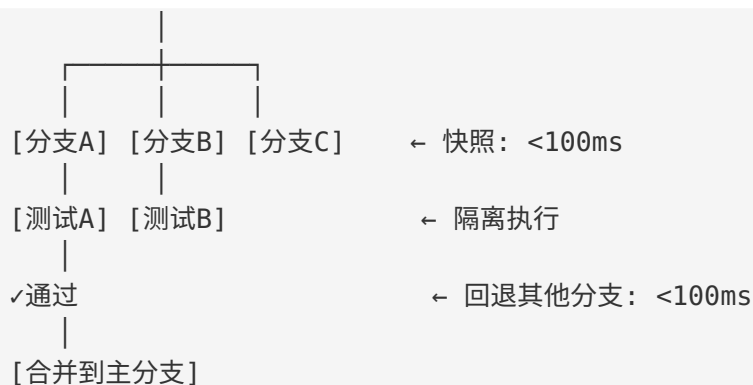
→ 像 ensemble 一样, 多个弱信号合成强信号

最有前途的方向: 专用 Value Network。想象一个在数百万代码修改及其结果 (测试通过/失败/性能变化/bug 引入) 上训练的模型, 它能直接输出“这个修改有 X% 的概率是正确的”。这就是编程领域的 value network, 也是 AlphaGo 成功的核心组件。

6.6.3 条件三: 高效的状态管理

理想的代码搜索状态管理:

[主分支]



需要的技术:

1. 文件系统快照: Copy-on-Write, ZFS, 或容器化
2. 进程隔离: 每个分支在独立容器/sandbox 中运行
3. 状态序列化: 快速保存/恢复完整开发环境
4. 增量编译: 只重新编译改变的部分

现有方案:

- Git worktree: 文件级隔离, 但不隔离进程
- Docker: 完整隔离, 但启动慢 (~2-5s)
- microVM (Firecracker): 快速启动 (~125ms), 完整隔离 ★
- Nix/Guix: 可重现的环境管理

预判: 微虚拟机 (microVM) + Copy-on-Write 文件系统的组合可能是最终答案。它可以在 ~100ms 内创建完整的代码执行环境副本, 这使得搜索树的分支成本变得可接受。

6.6.4 条件四: Search-Augmented Training

AlphaGo 成功的一个核心要素是**自我对弈 (self-play)** ——模型通过与自己对弈生成训练数据, 这些数据又用来改进模型, 形成正向循环。

编程领域需要类似的机制:

Search-Augmented Training 循环:

1. 基础 LLM 作为 Policy + Value Network



2. 使用 MCTS 在大量编程任务上搜索



3. 搜索过程产生数据:
 - (状态, 最优动作) → 训练 policy
 - (状态, 最终结果) → 训练 value

↓
4. 用新数据 fine-tune LLM

↓
5. 改进后的 LLM 作为更好的 Policy + Value Network

↓
回到 Step 2, 循环...

类比 AlphaGo:

- AlphaGo: self-play → 训练数据 → 更强的网络 → 更好的 self-play
 - 编程 Agent: MCTS 搜索 → 训练数据 → 更强的 LLM → 更好的搜索
-

6.6.5 RL-from-Code-Execution 假说

更进一步，我们可以设想一个完全的强化学习框架：

RL-from-Code-Execution:

- State: 代码仓库的当前状态
- Action: 代码修改（由 LLM 生成）
- Reward: 测试通过 (+1), 测试失败 (-0.5), 编译错误 (-1), 性能提升 (+0.3), ...
- Policy: LLM（选择下一步操作）
- Value: LLM 或专用模型（估计当前状态价值）

与传统 RLHF 的区别：

- RLHF: 人类反馈 → 主观、昂贵、缓慢
- RL-from-Code-Execution: 代码执行反馈 → 客观、免费、快速

这是编程领域独特的优势：代码可以自动执行和验证，这使得 RL 的 reward signal 比自然语言任务更容易获得。

这个假说的关键优势：编程可能是所有 LLM 应用领域中，最适合 RL 的领域——因为代码执行提供了自动化的、客观的反馈信号。这是自然语言对话、创意写作等领域所不具备的。

6.6.6 时间线预判

编程 Agent 的 AlphaGo 时刻路线图（推测）：

2024-2025：基础设施准备期

- ✓ 推理成本下降到可接受范围
- ✓ LATS/SWE-Search 等方法验证了可行性
- 容器化状态管理开始成熟
- 代码评估模型开始出现

2025-2026：早期系统涌现

- 专用代码 value network 出现
- Search-augmented training 开始被大实验室采用
- 异步搜索型 Agent 产品出现（非实时，但更准确）
- SWE-bench Full 达到 70%+

2026-2028：突破性时刻

- 搜索型 Agent 在复杂任务上明显超越人类开发者
- RL-from-Code-Execution 进入大规模实验
- "编程 Agent 的 AlphaGo 时刻" 可能在此窗口

不确定性因素：

- 模型能力的 scaling 是否持续？
 - 评估函数的精度能否突破？
 - 商业需求是否驱动足够的研发投入？
 - 监管环境是否友好？
-
-

6.6.7 为什么这件事意义重大

当前的贪心编程 Agent 已经在改变软件开发——但它们本质上是“辅助工具”，依赖人类设定方向。

一个具备 MCTS 级别搜索能力的编程 Agent 将是质的飞跃：

贪心 Agent（当前）：

- 人类："修复这个 bug"
- Agent：尝试一种方法 → 成功或失败
- 能力：一个中等水平程序员的速度和准确度

搜索型 Agent（未来）：

- 人类："修复这个 bug"
- Agent：系统性探索 50 种方法 → 统计收敛到最优解
- 能力：一个顶尖程序员的准确度，100 倍的速度

再下一步：

- 人类："实现这个产品"（高层规约）
- Agent：架构搜索 → 模块搜索 → 实现搜索 → 多层 MCTS
- 能力：一个顶尖工程团队的水平，持续 24/7 运作

从“good”到“superhuman”：贪心策略的上限是 LLM 单次推理的能力上限——大约等于一个优秀程序员。搜索策略的上限则取决于计算预算——理论上没有天花板。这就是 AlphaGo 的启示：搜索可以将 AI 从“人类水平”推到“超人水平”。

关键论文导读

必读论文

1. LATS: Language Agent Tree Search (Zhou et al., 2023)

论文：Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models

核心贡献： - 首次将 MCTS 框架完整应用到 LLM Agent - 证明了 LLM 可以同时充当 policy network、value network 和 simulation engine - 在多个 benchmark 上显著超越 ReAct 和 Reflexion

阅读重点： - Section 3: LATS 算法的完整描述（对照本模块 6.3 节理解） - Section 4.1: UCB 公式在 LLM Agent 中的适配 - Table 1-3: 与 ReAct/Reflexion/ToT 的对比实验 - Figure 2: 搜索树的可视化

批判性思考： - LATS 的 LLM-as-value-function 精度够用吗？对比 AlphaGo 的专用 value network。 - Token 消耗增加 10-50 倍，在什么场景下是值得的？ - 论文中的 benchmark 是否代表真实编程任务的复杂度？

2. SWE-Search (Antonis et al., 2024)

论文：SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement

核心贡献： - 将 MCTS 专门适配到软件工程场景（SWE-bench） - 引入了 Agent 操作级别的搜索粒度（而非 token 级别） - 展示了反思信息在搜索树分支间的传播机制

阅读重点： - Section 3: 如何将 SWE 任务建模为搜索问题 - Section 3.2: Value function 的设计——LLM 如何评估中间状态 - Section 4: 在 SWE-bench 上的实验结果 - 对比分析：MCTS 在“简单”和“困难”任务上的差异化收益

批判性思考： - 搜索粒度的选择（Agent 操作 vs. 代码行 vs. 文件）如何影响效果？ - 反思传播是否引入了 bias？ 一个分支的失败经验可能误导其他分支。 - 在真实代码库（百万行级别）上的可扩展性？

3. CodeTree (Li et al., 2024)

论文： CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models

核心贡献： - 多 Agent 架构的树搜索（Thinker/Coder/Critic 分工） - 统一的搜索框架支持多种策略（BFS/DFS/Best-First） - 在竞赛级编程题上展示了显著效果

阅读重点： - Section 3: 三个 Agent 的角色定义和协作机制 - Section 3.3: Critic Agent 的评估策略 - Section 4: 不同搜索策略（BFS vs DFS vs Best-First）的对比 - Table 2-3: 不同模型（GPT-4, Claude 3 等）下的效果

批判性思考： - 多 Agent 架构的 overhead 是否值得？ 对比单 Agent + MCTS。 - Critic Agent 和 Value Network 的本质区别是什么？ - 从竞赛编程到实际软件工程的 gap 有多大？

延伸阅读

论文	关键点	与本模块的关联
Tree of Thoughts (Yao 2023)	LLM 推理的树搜索基础	Level 2 的理论基础
Reflexion (Shinn 2023)	自我反思与线性回溯	Level 1 的代表工作
AlphaCode (Li 2022)	大规模采样 + 过滤	搜索 vs 过滤的区别
ReAct (Yao 2023)	Agent 的 Reasoning + Acting	Level 0 的理论基础
AlphaGo (Silver 2016)	MCTS + 深度学习	6.3 节的原始来源
AlphaZero (Silver 2017)	纯自我对弈 + MCTS	6.6.4 节的灵感来源

实操环节：贪心 vs 树搜索对比实验

实验目标

通过一个具体的 bug 修复任务，对比贪心策略和树搜索策略的行为差异，直观理解搜索策略的价值。

实验设计

任务描述

我们设计一个“有陷阱的 bug”——表面上看最可能的原因是 A，但实际原因是 B。这种 bug 会让贪心策略陷入错误方向，而树搜索有机会探索到正确方向。

```
# bug_scenario.py - 一个有"陷阱"的 bug 场景

class OrderProcessor:
    """订单处理器 - bug: 某些订单的折扣计算错误"""

    def __init__(self, db):
        self.db = db

    def calculate_discount(self, order):
        """计算订单折扣"""
        customer = self.db.get_customer(order.customer_id)

        # 陷阱：看起来像是这里的逻辑问题（实际不是）
        if customer.is_vip:
            base_discount = 0.15
        else:
            base_discount = 0.05

        # 真正的 bug：季节性折扣函数有问题
        seasonal = self._get_seasonal_discount(order.date)

        # 陷阱：看起来可能是叠加逻辑的问题（实际不是）
        total_discount = min(base_discount + seasonal, 0.30)
        return total_discount

    def _get_seasonal_discount(self, date):
        """季节性折扣 - 真正的 bug 在这里"""
        month = date.month
        # Bug: 月份边界条件错误
        # 应该是 month >= 11 or month <= 2 (冬季: 11,12,1,2月)
        # 实际写成了 month >= 11 and month <= 2 (永远为 False!)
```

```

    if month >= 11 and month <= 2: # ← 这是 bug!
        return 0.10
    elif 6 <= month <= 8:
        return 0.05
    return 0.0

def process_order(self, order):
    """处理订单"""
    discount = self.calculate_discount(order)
    final_price = order.total * (1 - discount)
    return final_price

```

test_orders.py – 测试用例

```

def test_winter_discount():
    """冬季订单应该有 10% 季节性折扣"""
    db = MockDB(customer=Customer(is_vip=False))
    processor = OrderProcessor(db)
    order = Order(customer_id=1, total=100, date=date(2024, 12, 15))
    # 期望: 5% base + 10% seasonal = 15% discount → $85
    assert processor.process_order(order) == 85.0 # FAILS! 得到 $95

def test_summer_discount():
    """夏季订单应该有 5% 季节性折扣"""
    db = MockDB(customer=Customer(is_vip=False))
    processor = OrderProcessor(db)
    order = Order(customer_id=1, total=100, date=date(2024, 7, 15))
    assert processor.process_order(order) == 90.0 # PASSES

def test_vip_winter():
    """VIP 冬季订单"""
    db = MockDB(customer=Customer(is_vip=True))
    processor = OrderProcessor(db)
    order = Order(customer_id=1, total=100, date=date(2024, 1, 15))
    # 期望: 15% base + 10% seasonal = 25% discount → $75
    assert processor.process_order(order) == 75.0 # FAILS! 得到 $85

```

策略 A: 模拟贪心搜索

指导学生按照贪心 Agent 的行为模式操作:

Step 1: 看到测试失败信息

```
"test_winter_discount FAILED: expected 85.0, got 95.0"
```

```
"test_vip_winter FAILED: expected 75.0, got 85.0"
```

Step 2: 第一直觉 – 折扣计算逻辑有误

- 检查 `calculate_discount` 方法
- 检查 VIP 判断逻辑（看起来正确）
- 检查折扣叠加逻辑（看起来正确）
- 陷入困境...

Step 3: 继续在同一方向挖掘

- 修改 `base_discount` 值？不对...
- 修改 `min()` 上限？不对...
- 检查 MockDB 的 `is_vip` 设置？正确...

Step 4: 可能尝试 3-5 次修改后才发现

- 或者运气好，最终找到 `_get_seasonal_discount`
- 或者运气不好，超时失败

贪心路径记录：

- 尝试 1: 修改 `base_discount` → 测试失败（方向错误）
 - 尝试 2: 修改 `min()` 逻辑 → 测试失败（方向错误）
 - 尝试 3: 检查 `customer` 逻辑 → 无发现（方向错误）
 - 尝试 4: 终于检查 `seasonal` 函数 → 发现 bug
- 总步数：4，浪费步数：3
-
-

策略 B：模拟树搜索

指导学生按照树搜索的行为模式操作：

Step 1: 分析失败测试，生成多个假设（Expansion）

- 假设 A: `base_discount` 计算逻辑错误（概率：30%）
- 假设 B: `seasonal_discount` 计算错误（概率：40%）
- 假设 C: 折扣叠加/上限逻辑错误（概率：20%）
- 假设 D: 测试用例本身有问题（概率：10%）

Step 2: 并行评估每个假设（Simulation）

假设 A 评估：

- 检查 `base_discount`: `VIP=0.15`, 非VIP=`0.05` ✓
- 测试结果：差值恰好是 10%，与 `base` 无关
- 评分：0.2（不太可能）

假设 B 评估：

- 检查 `_get_seasonal_discount`
- 发现: `month >= 11 and month <= 2` → 永远为 `False!`
- 解释了为什么冬季折扣 = 0（缺少 10%）
- 评分：0.95（极可能!）★

假设 C 评估：

- $\min(x, 0.30)$ 逻辑合理
- 但差值恰好是 10%，非上限问题
- 评分：0.1（不太可能）

Step 3: 选择最高评分的假设，深入探索 (Selection)

- 选择假设 B，修复 and → or
- 运行测试 → 全部通过！✓

树搜索路径记录：

- 分支 1（假设 A）：评估 → 排除（0.2）
 - 分支 2（假设 B）：评估 → 确认 → 修复 → 成功！
 - 分支 3（假设 C）：评估 → 排除（0.1）
 - 有效步数：1（直接找到 bug）
 - 总评估数：3（但更有系统性）
-

实验步骤

Step 1: 设置环境 (5 分钟)

1. 创建上述 `bug_scenario.py` 和 `test_orders.py` 文件
2. 补全必要的数据类（`Order`，`Customer`，`MockDB`）
3. 确认测试运行，验证两个测试确实失败

Step 2: 贪心策略体验 (15 分钟)

1. **规则：**你是一个贪心 Agent，每一步只能选一个行动
2. 从阅读测试失败信息开始
3. 按直觉选择最可能的原因，尝试修复
4. 记录每次尝试的：
 - 假设是什么
 - 做了什么修改
 - 结果是什么（测试通过/失败）
 - 消耗了多少“步数”

Step 3: 树搜索策略体验 (15 分钟)

1. **规则：**你是一个树搜索 Agent，先生成所有可能的假设，再逐一评估
2. 从分析失败模式开始：哪些测试失败了？差值是多少？
3. 列出 3-5 个独立的假设

- 4. 对每个假设进行快速评估（不实际修改代码，只做分析）
- 5. 选择评分最高的假设进行实际修复
- 6. 记录整个搜索过程

Step 4: 对比分析 (15 分钟)

填写以下对比表格：

维度	贪心策略	树搜索策略
总步数		
无效尝试数		
找到根因的步数		
修复的信心度		
是否掉入“陷阱”		
总体效率评价		

Step 5: 使用 LLM 重现 (20 分钟)

用实际的 LLM（如 Claude 或 GPT-4）进行两轮实验：

实验 A — 贪心 Prompt：

这段代码有 bug，请修复。
[粘贴代码和测试失败信息]
请一步一步分析并修复。

实验 B — 树搜索 Prompt：

这段代码有 bug。请按以下步骤分析：
1. 首先，列出 3-5 个可能的 bug 原因（不要急着修复）
2. 对每个原因进行独立评估，给出概率评分
3. 选择概率最高的原因进行修复
4. 验证修复是否解决了所有失败的测试

[粘贴代码和测试失败信息]

记录两种 prompt 下 LLM 的行为差异。

Step 6: 进阶思考 (10 分钟)

回答以下问题：1. 在这个简单的例子中，树搜索的优势有多大？如果 bug 更复杂呢？2. 树搜索需要“评估每个假设”——如果评估本身不准确怎么办？3. 如果你要设计一个真正的 MCTS 编程 Agent，最大的工程挑战是什么？4. 贪心策略通过什么方式可以部分弥补它的搜索缺陷？（提示：思考更强的 LLM、更好的 prompt、更丰富的上下文）

预期观察

通过这个实验，学生应该能够直观理解：

- 贪心的脆弱性：**当“第一直觉”是错误的时候，贪心策略会浪费大量资源
 - 搜索的系统性：**通过显式列举和评估假设，树搜索更不容易被“陷阱”误导
 - 评估函数的重要性：**树搜索的效果高度依赖于“假设评估”的质量
 - Prompt 即策略：**在当前 LLM Agent 中，prompt 的设计等价于搜索策略的选择
 - 成本 trade-off：**树搜索需要更多“评估”工作，在简单任务中可能是过度投入
-

本模块小结

核心观点回顾

- 搜索策略光谱：**从贪心（Level 0）到 MCTS（Level 3），编程 Agent 的搜索能力有四个清晰的等级。当前主流产品处于 Level 0。
- 贪心统治的原因：**Token 成本、延迟要求、状态管理复杂度和“够用”原则共同维持了贪心策略的统治地位。
- AlphaGo 的启示：**MCTS + Policy Network + Value Network 的组合证明了搜索可以将 AI 从“人类水平”推到“超人水平”。
- 代码空间的特殊困难：**无界状态空间、模糊评估函数、非局部效应等使得 MCTS 在代码领域需要本质性的改造。
- 前沿进展：**LATS、SWE-Search、CodeTree 等工作正在开拓从贪心到搜索的路径，但仍面临成本、评估精度和泛化性的挑战。
- AlphaGo 时刻的条件：**更便宜的推理、更好的评估函数、高效的状态管理、search-augmented training——这些条件正在逐步成熟。

一句话总结

当前编程 Agent 处于“搜索策略的石器时代”。从贪心到 MCTS 的进化不是“是否”的问题，而是“何时”的问题——而这个“何时”可能比大多数人预期的更早。

思考题

基础题

- 策略识别**：分析你日常使用的编程 Agent（如 Cursor、Copilot、Claude Code），它属于搜索光谱的哪个等级？给出具体的行为证据。
- UCB 直觉**：在 UCB 公式中，探索常数 c 的大小如何影响搜索行为？ $c=0$ 和 $c=\infty$ 分别对应什么策略？
- 成本计算**：假设一个编程任务平均需要 50K tokens（贪心策略）。如果采用分支因子 3、深度 5 的树搜索（每个节点需要额外 5K tokens 用于评估），总 token 消耗是多少？是贪心的多少倍？

进阶题

- 评估函数设计**：如果你要为编程 Agent 设计一个 value function，你会融合哪些信号？如何确定各信号的权重？是否有方法自动学习这些权重？
- 状态管理方案**：设计一个支持代码搜索树分支/回退的状态管理系统。考虑文件变更、依赖安装、数据库迁移等不同类型的操作。画出系统架构图。
- 搜索粒度选择**：在代码 MCTS 中，搜索树的“一步”应该是什么粒度？字符级修改？行级修改？函数级修改？文件级操作？讨论各粒度的优缺点。

开放题

- 反驳论点**：有人认为“随着 LLM 能力的提升，单次推理（贪心策略）最终会和搜索策略一样好，因此不需要搜索”。你同意还是反对？给出论证。（提示：思考 AlphaGo 的历史——即使单次评估模型变强了，搜索仍然提供了额外价值。）
- 商业模式**：如果搜索型编程 Agent 的成本是贪心型的 10-50 倍，应该如何设计商业模式？“按结果收费”而非“按 token 收费”是否可行？

3. **安全与对齐：**一个具备 MCTS 搜索能力的编程 Agent 可能探索出人类意想不到的代码修改路径。这带来什么安全风险？如何确保搜索过程“对齐”于人类意图？
4. **未来预测：**你认为“编程 Agent 的 AlphaGo 时刻”最可能以什么形式出现？是一个在 SWE-bench 上达到 95%+ 的系统？还是一个能独立完成完整软件项目的 Agent？还是其他形式？给出你的预测和理由。